Architecture des Systèmes d'Exploitation : Processus, Cycle de Vie et Signaux POSIX

Thème principal : Gestion des processus et communication inter-processus sous Unix/Linux

Synthèse exécutive : Ce cours pose les fondements de l'exécution logicielle dans un système d'exploitation de type Unix/Linux. Il détaille le passage d'un programme statique résidant sur le disque dur à une instance dynamique en mémoire RAM (le processus). Pour gérer l'exécution concurrente de multiples applications sur un matériel limité, le noyau s'appuie sur le multitâche préemptif et un cycle de vie structuré autour de trois états (Élu, Prêt, Bloqué). L'analyse aborde ensuite la structure interne du processus via le Bloc de Contrôle du Processus (PCB), l'arborescence généalogique stricte issue de systemd (PID 1), le mécanisme de duplication et d'exécution (fork/exec), ainsi que l'administration asynchrone des tâches au moyen des signaux POSIX (SIGINT, SIGTERM, SIGKILL, SIGSTOP, SIGCONT).

Du Programme Statique au Processus Dynamique : Cycle de Vie, Attributs et Multitâche

Pour qu'une application s'exécute, le système d'exploitation doit faire basculer son code binaire d'un état statique sur le disque dur vers une entité active en mémoire vive (RAM) appelée processus. Dans un système moderne où le nombre de tâches dépasse le nombre de cœurs processeurs physiques, le noyau Linux met en œuvre le multitâche préemptif. L'ordonnanceur (scheduler) alloue des tranches de temps extrêmement brèves (quanta) à chaque processus, simulant ainsi une exécution simultanée. Le cycle de vie d'un processus s'organise selon trois états majeurs : 1. **Élu (Running)** : Le processus occupe physiquement un cœur processeur et exécute ses instructions machine. 2. **Prêt (Ready)** : Le processus possède toutes ses ressources en mémoire RAM et attend que l'ordonnanceur lui attribue son prochain quantum CPU. 3. **Bloqué (Waiting / Blocked)** : Le processus est suspendu dans l'attente d'une opération d'Entrée/Sortie (lecture disque, saisie clavier, paquet réseau). Le noyau libère le processeur au profit d'une autre tâche. Lorsque l'événement attendu survient, le processus ne repasse **jamais** directement à l'état Élu : il bascule impérativement dans l'état Prêt afin d'être réinscrit dans la file de planification.

- Distinction Programme vs Processus : Un programme est un fichier exécutable passif rangé sur disque ; un processus est une instance dynamique active chargée en RAM.
- Multitâche Préemptif : Découpage du temps processeur par l'ordonnanceur (scheduler) en quanta de temps attribués successivement aux processus.
- Bloc de Contrôle du Processus (PCB) : Structure interne de données conservant l'état du processeur, registres, carte mémoire, identifiants (PID, PPID, UID) et descripteurs de fichiers.
- Modèle des 3 États : Élu (Running), Prêt (Ready), Bloqué (Waiting/Blocked). Un processus sortant de l'état Bloqué repasse obligatoirement par Prêt.

Cas concret enseignant : *Analogie de la recette de cuisine : Le programme est le livre de recettes rangé dans l'armoire (passif, ne consomme pas d'ingrédients ni d'électricité). Le processus est le cuisinier en train de réaliser activement la recette (dynamique, occupe du temps, de l'espace sur le plan de travail en RAM et manipule des ustensiles correspondant aux descripteurs de fichiers).*

Filiation, Arborescence et Mécanismes de Création (fork et exec)

Sous Unix/Linux, aucun processus ne naît spontanément. Tout processus dérive d'une arborescence généalogique unique dont la racine est le premier processus démarré par le noyau lors du boot : systemd (ou init), qui porte de façon constante le PID 1. Lorsqu'un utilisateur lance une commande dans un shell Bash (processus parent), la création de la tâche s'effectue en deux phases distinctes : 1. **fork()** : Le processus père exécute cet appel système. Le noyau duplique entièrement le père. L'enfant hérite d'une copie conforme de l'espace mémoire, reçoit

son propre PID et voit son PPID réglé sur le PID du père. 2. **exec()** : Immédiatement après la bifurcation, l'enfant appelle une fonction de la famille exec(). Le noyau remplace le code hérité par le binaire du nouveau programme (par exemple `/usr/bin/ls`). Le PID est conservé mais le code exécuté est complètement renouvelé.

- Arborescence et Identifiants (PID / PPID) : Chaque processus possède un PID unique et référence le PID de son père (PPID). La racine absolue est systemd / init (PID 1).
- Appel système fork() : Duplique le processus parent à l'identique pour créer un processus fils disposant de son propre PID.
- Appel système exec() : Remplace le code et les données du processus appelant par l'image binaire d'un nouveau programme.

Contrôle des Tâches et Communication Inter-Processus : Les Signaux POSIX

Les signaux POSIX constituent le mécanisme d'interruption logicielle asynchrone fondamental sous Linux. À la réception d'un signal, un processus peut adopter l'un des trois comportements suivants : 1. Exécuter l'action par défaut du système (généralement la fin du processus). 2. Ignorer le signal. 3. Intercepter le signal en déclenchant un gestionnaire personnalisé (signal handler). Afin de garantir l'autorité d'administration sur des processus bloqués ou défaillants, la norme POSIX réserve deux signaux incapturables : **SIGKILL (Signal 9)** et **SIGSTOP (Signal 19)**. Traités directement par le noyau, ils ne peuvent jamais être détournés ou bloqués par le code utilisateur.

- Signal POSIX : Interruption logicielle asynchrone transmise par le noyau à un processus pour signaler un événement.
- Gestionnaire de signal (Signal Handler) : Routine enregistrée par l'application pour intercepter (catch) un signal au lieu d'exécuter l'action par défaut.
- SIGINT (Signal 2) : Déclenché par Ctrl+C, demande une interruption douce.
- SIGTERM (Signal 15) : Demande d'arrêt propre envoyée par default par 'kill '.
- SIGKILL (Signal 9) : Ordre d'extinction inconditionnel direct du noyau. Impossible à capturer ou ignorer.
- Job Control (SIGSTOP 19 / SIGCONT 18) : Ctrl+Z suspend la tâche (SIGSTOP) ; 'fg' ou 'bg' relance la tâche (SIGCONT).

Cas concret enseignant : *Analogie de l'expulsion : SIGTERM (Signal 15) est le gardien d'immeuble qui frappe à la porte pour vous demander de ranger vos affaires et de sortir proprement. SIGKILL (Signal 9) est la brigade d'urgence qui défonce la porte et vous fait sortir immédiatement sans vous laisser le temps de toucher à quoi que ce soit.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la commande 'kill 9 1234' renvoie-t-elle une erreur dans le terminal Linux alors que la commande 'kill -9 1234' fonctionne parfaitement ?
Réponse du professeur : Dans la syntaxe du shell Bash, le tiret '-' indique que le paramètre est une option spécifiant le numéro du signal ('-9' pour SIGKILL). Sans tiret, la commande 'kill' interprète '9' et '1234' comme deux arguments identifiants de processus (PID). Elle tente donc d'envoyer le signal par défaut SIGTERM au PID 9 (ce qui échoue faute de privilèges) et au PID 1234.

Question étudiant : Qu'est-ce qu'un processus dit 'Zombie' et comment peut-on le détruire avec la commande kill ?
Réponse du professeur : Un processus Zombie est une tâche terminée qui a libéré sa mémoire RAM et ses ressources processeur, mais dont l'entrée subsiste dans la table des processus du noyau parce que son processus père n'a pas encore lu son code de fin via l'appel système wait(). Il est impossible de détruire un Zombie directement avec la commande 'kill' (même avec SIGKILL -9) car le processus est déjà mort. Pour le faire disparaître, il faut soit forcer le père à exécuter wait(), soit tuer le processus père : le Zombie est alors adopté par systemd (PID 1) qui nettoie immédiatement l'entrée.

■ Exercice & Démonstration d'Application

Énoncé : Dans un terminal Bash sous Linux, un administrateur effectue la séquence de commandes suivante : 1. sleep 600 & 2. ps -ef | grep sleep 3. kill -15 Explicitez le rôle du symbole '&', détaillez le mécanisme du signal 15 et

décrivez l'état final du processus.

Démarche de résolution :
Étape 1 : Le symbole '&' placé à la fin de la commande 'sleep 600 &' demande au shell d'exécuter le processus en arrière-plan (background). Le shell affiche l'identifiant du job et le PID attribué par le noyau (ex: 4521) puis rend immédiatement la main à l'utilisateur. Étape 2 : La commande 'ps -ef | grep sleep' liste les processus du système et permet de retrouver le PID (4521) et le PPID (PID du shell Bash). Étape 3 : La commande 'kill -15 4521' envoie le signal SIGTERM (Signal 15) au processus 4521. N'ayant pas de gestionnaire spécifique pour capturer ce signal, l'application exécute l'action par défaut : l'arrêt propre et la libération de ses structures mémoire.

```
#!/bin/bash
# Séquence de commandes de gestion de processus

// TRAITEMENTS
sleep 600 &
ps -ef | grep sleep
kill -15 4521
```

Résultat : Le processus 'sleep 600' est proprement terminé. Une vérification avec 'ps -ef | grep sleep' confirme sa disparition définitive de la table des processus.

■ Auto-Évaluation Flash d'Examen

Q : Quel est le nom et le PID du tout premier processus utilisateur démarré par le noyau Linux ?

R : Le tout premier processus utilisateur est 'systemd' (ou 'init' sur les anciens systèmes) et son PID est impérativement 1.

Q : Quel signal POSIX est généré lorsqu'un utilisateur presse la combinaison de touches 'Ctrl + C' dans son terminal ?

R : La combinaison 'Ctrl + C' génère le signal SIGINT (Signal 2).

Q : Pourquoi le signal SIGKILL (Signal 9) ne peut-il jamais être intercepté par un gestionnaire de signal développé dans une application ?

R : SIGKILL est traité directement par le noyau Linux lui-même. La norme POSIX interdit l'interception ou le masquage de ce signal afin de garantir à l'administrateur la capacité absolue d'arrêter n'importe quel processus défaillant.